



Universidad Católica de Chile

FACULTAD DE MATEMÁTICAS - ESCUELA DE INGENIERÍA  
INSTITUTO DE INGENIERÍA MATEMÁTICA Y COMPUTACIONAL

# TAMIZAJE DE ATRESIA BILIAR

MANUAL DE USUARIO

*IMT3500 - Proyecto Final*  
*Taller de Ingeniería Matemática*

Autores:

Martín Alcántara  
Nicolás Donoso

2026-1

# 1 Manual de Usuario: Integrar tamizaje de atresia biliar a aplicación móvil.

Objetivo: Integrar la funcionalidad del módulo "biliary\_atresia" a una aplicación móvil. Esta debe entregar una imagen, y se le retornará un valor en  $[0, 1]$ .

En esta guía se detalla cada archivo en detalle, como utilizarlos y manejarlos en una instancia de desarrollo de software.

## 1.1 Arquitectura recomendada de aplicación.

El pipeline tiene como dependencias las librerías OpenCV, SciPy, Scikit-Learn y PyTorch. Si bien es posible hacer estos códigos funcionar localmente, esto no es recomendado debido al soporte limitado que tienen en Android/iOS.

En su lugar, se recomienda que el dispositivo móvil, una vez obtenida la imagen, se conecte al servidor enviando la imagen, y las operaciones de backend sean realizadas por el lado de servidor, el cual retornaría únicamente el resultado por interpretar antes de finalizar la conexión.

En otras palabras, el trabajo de la aplicación sería obtener la imagen, ya sea por captura o carga manual de ella, y el servidor se encargaría de todo el trabajo descrito en el presente documento.

## 1.2 La función más importante

A modo general, toda la maquinaria se reduce a la función "persistence.predict\_single":

```
1 from biliary_atresia import load_model
2 from biliary_atresia.persistence import predict_single
3
4 # Load once at process startup, not per-request
5 model = load_model(name="risk_model_v1", directory="checkpoints")
6
7 def handle_upload(image_path: str) -> dict:
8     result = predict_single(
9         image_path,
10        feature_method="avg",
11        model=model,           # reuse the loaded model; skips disk I/O per request
12        p_safe=0.25,
13        p_risk=0.75,
14    )
15    return {
16        "probability": result["probability"], # float in [0, 1] - send this to the app
17        "label": result["label"],             # 0: safe / 1: unsure / 2: at risk
18        "label_str": result["label_str"],
19    }
```

Se define un wrapper tipo REST mínimo para ilustrar el modo de uso principal. La función "handle\_upload" representa, entonces, todo el backend que la aplicación necesitaría.

Luego, la aplicación puede: entregar la probabilidad estimada de manera directa, o más recomendado es retornar la categoría a la que pertenece, posiblemente con un mensaje de explicación. Por ejemplo "su bebé se encuentra sano", "se recomienda volver a tomar una imagen de la siguiente deposición", y "se recomienda asistir a un médico: descargar imagen", para los casos respectivos.

## 1.3 Desglose de los distintos archivos.

### 1.3.1 "imaging.py" - Archivo de servidor; no es para uso local

"segment\_image" es llamado por "predict\_single", por lo que no es necesario llamarlo en la aplicación. No obstante, se recuerda que el desempeño dependerá de la calidad de la imagen. En particular, se espera:

- Iluminación regular y consistente.
- Fondo blanco, idealmente el pañal con la deposición.
- Imágenes centradas y bien enfocadas.

Por lo anterior, se recomienda enfocar un trabajo en asegurar, en tiempo real, que la imagen sea bien tomada, con alguna interfaz gráfica. Se recomienda en particular YOLO, una vez obtenidos los suficientes datos para su uso. Esto permite definir una cubierta o guía de referencia para el usuario, y asegura el correcto funcionamiento en adelante.

### 1.3.2 "features.py" - Archivo de servidor; no es para uso local

La función "predict\_single" necesita un vector "(3,)" LAB para el modelo. Dentro de los métodos de la función, "avg" y "gauss", entregan esto. "conv" retorna múltiples copias de esto con el fin de simular más datos. "cluster" requiere trabajo adicional, en la forma de una función de agregación, de la cual se sugiere tomar el máximo (i.e. evaluar el peor caso), pero se debe explorar. Se debe usar "feature\_method='avg'" para toda clasificación individual, lo cual incluye el uso en aplicación, a menos que se tenga una buena razón para cambiar, dado que es el más simple y eficiente.

### 1.3.3 "model.py" - Archivo de servidor; no es para uso local

"RiskMLP" es una red pequeña de tamaño:  $3 \rightarrow 64 \rightarrow 32 \rightarrow 1$ . Por ende, es altamente eficiente y no debería traer problemas de cómputo. Se recomienda expandir esta estructura si el modelo no está aprendiendo de manera eficiente. En aquellos casos, es necesario monitorear su desempeño computacional. Se recomienda Optuna para Python para realizar el refinamiento de hiperparámetros.

### 1.3.4 "persistence.py" - Archivo de trabajo; Se llama para realizar las predicciones, y para actualizar los modelos cuando se tiene nueva data de entrenamiento

Cuando se busca actualizar el modelo, se realizan guardados parciales a lo largo de tres archivos: "i\_name.pt" (pesos del modelo), "i\_name.meta.json" (metadata), "i\_name\_data.npz" (data de entrenamiento usado para producir el modelo). Por ende, se recomienda:

- Asignarle nombre y número de versión. Por ejemplo, usar nombre como "risk\_model.v3", en lugar de reemplazar un único archivo llamado "modelo".
- Dejar un registro de las decisiones tomadas para la construcción del modelo. Fecha de entrenamiento, particiones de los datos, datos excluidos e incluidos, y métodos utilizados. De esta manera, se puede devolver a una versión estable ante cualquier error.
- Se debe actualizar el modelo de manera online, o de manera rutinaria. No se debe realizar cada vez que se haga una predicción, o cada vez que se obtenga un dato en tiempo real. En especial dado que, con la construcción actual, se reentrena desde cero. Obteniendo un buen y estable desempeño, con suficientes datos, esto último debe ser modificado para que las actualizaciones rutinarias impliquen únicamente un finetuneo.

### 1.3.5 "viz.py" - Archivo de servidor; no es para uso local

Este archivo es un helper para visualización en desarrollo. Permite visualizar los resultados con el fin de debuggear y mostrar resultados preliminares. No debe llamarse en otros escenarios.

## 1.4 Errores conocidos

### 1.4.1 "extract\_lab\_feature(..., method='cluster')"

Este método retorna un diccionario con más información que la que esperan las funciones que llaman a "extract\_lab.feature". Se necesita incorporar su funcionalidad antes de usarse. Se recomienda usar una función de agregación, como el máximo o el mínimo. Esto es, se realiza la predicción para cada color, y se trabaja con el máximo, o mínimo. También se puede usar el promedio ponderado a la masa del cluster. Este método se debe usar cuando se detectan múltiples colores suficientemente distintos en la imagen. Pero en la practica no encontramos ninguno con estas características en los datos que obtuvimos.

### 1.4.2 "extract\_lab\_feature(..., method='conv')"

Cuando la imagen es muy pequeña, "window" es también muy pequeño. Por ende, puede no tener suficiente tamaño para crear los patches según el umbral definido. En este caso, retornará arrays vacíos sin entregar error. Y luego, la probabilidad será NaN, que es interpretado como "unsure", independiente de la naturaleza del dato. Dado que "conv" espera ser usado únicamente en entrenamiento de versiones temprana del modelo, se recomienda eliminar esos datos, o simplemente usar "avg" para salvar un único dato.

### 1.4.3 "contour\_largest"

Existen casos bordes en donde el algoritmo no puede distinguir los depósitos con el fondo, en especial cuando la foto no está bien tomada. Esto se espera que se arregle con YOLO, pero por sanidad se recomienda implementar algo por el estilo de:

```
1 img_lab, big_mask, _ = segment_image(img_rgb)
2 foreground_fraction = (big_mask > 0).mean()
3 if foreground_fraction < 0.01 or foreground_fraction > 0.9:
4     # Segmentation almost certainly failed - reject rather than run inference on garbage
5     raise ValueError("Could not isolate a subject in this photo")
```

Y luego indicar, a través de la aplicación, que se requiere volver a tomar la imagen. Esto permite identificar cuando el fondo represente mucho porcentaje de la máscara, o una fracción muy pequeña.

Adicional a esto, existe un segundo problema acerca de la dirección del threshold para la dimensión  $a$ . Algunos casos requieren  $a > \tau$ , mientras que otros  $a < \tau$ . Para distinguir esto, se propone escoger aquel que produzca una máscara más centrada. Al ser opuestas, se espera que la correcta sea similar a un disco, mientras que la incorrecta tenga un disco como orificio. De esta manera, cualquier medida de centralidad bastaría. Cabe destacar que este tipo de soluciones únicamente resulta cuando se tiene control mínimo de la calidad de la imagen, a traves de YOLO o preprocesamientos adecuados.

## 1.5 Checklist previo a la integración

El backend llama únicamente a "predict\_single(..., feature\_method='avg')".

No se permite modificación de parámetros al usuario.

La probabilidad nunca es "NaN".

Los checkpoints tienen nombres descriptivos y con versiones.

La aplicación soporta un control de calidad mínimo de la imagen, idealmente a través de YOLO.

El contrato de conexión con el servidor es un JSON que contiene únicamente la probabilidad y categoría.